ACN PROJECT · TEAM 7

Frame-aware multipath QUIC with a learned bandwidth predictor

Streaming live lectures over two cellular connections without missing playback deadlines

Kabir Bajaj and Team 7 · 29 October 2026

Slide 1. Introduce the team and the one-line idea: a phone with two SIM cards has two internet connections; we send a live lecture over both at once, and we built a scheduler, driven by a small neural network, that decides which connection each video frame takes so frames arrive on time.

The setting

- Students attend live classes on a **phone over mobile data**
- Many phones hold **two SIM cards**, two carriers: two internet connections at once
- Each one alone is often **slow and unsteady**: a few Mbit/s, 50–150 ms delay, short coverage holes
- **Multipath QUIC** runs one connection over both at the same time

THE QUESTION

A **scheduler** inside the transport decides, packet by packet, which connection carries the data. How should it decide when the data is **live video**?

Slide 2. Start from the user: a student in India on mobile data. Dual-SIM phones are common, so there are two connections. Multipath QUIC (an IETF draft about to become an RFC) lets one connection use both paths. The scheduler is the piece that decides which path each packet takes; that is what this project is about.

How a live lecture travels



I-frame: full picture, ~55 KB, every 2 s P-frame: only the changes, ~8 KB, 24 per second

- A frame is usable only when **every byte** has arrived
- The player shows frame *i* a fixed **playback delay** after it was produced (we use 300 ms)
- Later than that: a **missed deadline**, the picture freezes or glitches

Slide 3. Explain video frames: the encoder sends a big complete picture (I-frame) every 2 seconds and small difference frames in between. The player has a schedule; a frame that arrives after its slot is a missed deadline and the viewer sees a freeze. Big frames are the risky ones because they take longest to send.

The problem: schedulers see bytes, not frames

What standard schedulers do

- **MinRTT**: lowest-delay path with room
- **Round-robin**: alternate paths
- **Redundant**: everything on both paths
- **ECF**: wait for the fast path if that finishes sooner

Why that hurts video

- **Frames get split**: done only when the slow half arrives
- **Big frames block small ones** queued behind them

WORKED EXAMPLE

A 45 KB I-frame on a 1.5 Mbit/s path needs **240 ms** to transmit, plus 30 ms travel: **270 ms of a 300 ms budget**, before counting the frames already queued.

With two slow paths there is no room to spare.

Slide 4. None of the standard schedulers knows where a frame starts or ends or when it is due. Two failure modes: a frame split across a fast and a slow path finishes only when the slow part lands; and a big I-frame occupies a path while small frames wait behind it. The worked example shows how tight the budget is on low-bandwidth links.

Research questions

1. How often do transport-only multipath schedulers make lecture frames **miss their playback deadline** on two low-bandwidth cellular paths?
2. Can a scheduler that knows each frame's **size and deadline**, driven by a **learned prediction** of each path's capacity, do better?
3. Which part of the design makes the difference? (ablations)

Closest work: MoMQ (2026), frame-aware multipath QUIC for Starlink + Wi-Fi. Two similar low-bandwidth cellular paths are not studied.

Slide 5. State the three questions. Question 3 is answered with ablations: a hand-tuned version of our scheduler and a frame-pinning version without deadlines. Mention MoMQ as the closest related work and how our setting differs.

Our approach

Path observations
every 100 ms, per path: delivery rate, send rate, queueing delay, loss, cwnd, BBR estimate

→

Neural network
predicts a cautious capacity for the next 0.5 s

→

Deadline scheduler
picks the path(s) for each video frame

- Each frame on its **own QUIC stream**; the sender tells the transport its **deadline**
- Built **inside TQUIC** (Tencent's Rust QUIC library), not as a simulation

Slide 6. The big picture: every 100 ms we measure each path; a small neural network turns those measurements into a cautious estimate of how fast the path will be for the next half second; the scheduler uses that estimate to decide where each frame goes. All of it runs inside a real QUIC implementation.

The scheduler, per frame

$$T_p = (Q_p + S) / B_p + sRTT_p / 2$$

finish time on path p · S frame size · Q_p bytes already queued · B_p predicted capacity · $sRTT_p$ round-trip time

1. If a path finishes the frame before its deadline: **whole frame on the path that finishes first**
2. Otherwise (big I-frames): **split** so both parts finish at the same moment
3. Keep each frame on its path(s), so **small frames overtake** a big one on the other path
4. A frame already past its deadline may use any path (safety net)

Slide 7. Walk through the formula: time to drain what is ahead of the frame plus the frame itself at the path's rate, plus one-way delay. Rules 1-3 are the proposal's design; rule 4 we added after finding a failure where a path collapsed and frames pinned to it never arrived. The split solves a simple linear equation so both halves land together.

Why machine learning? The estimate B_p is the hard part

Simple estimates of a path's capacity fail in opposite ways:

- **BBR's estimate** and the **"best of last 3 s"** rule are often **too high** when a link fades
- The **recent delivery rate** is far **too low** on a path that carries little traffic
- Too high is what makes a frame **miss its deadline**

ON HELD-OUT CELLULAR TRACES

BBR overestimates capacity **38.2%** of the time

"Best of last 3 s" overestimates **47.1%**

Mean delivery rate is off by **72%** (median)

Slide 8. This is the motivation for ML. The scheduler's formula is simple physics; the uncertain part is the path's capacity in the near future. Hand-made estimates are either too optimistic on fading links or far too pessimistic on idle links. A learned model can combine all the signals and tell us how sure it is.

The learned bandwidth predictor

- **Input (44 numbers)**: for each 100 ms tick of the last second, delivery rate, send rate, queueing delay, loss ratio; plus cwnd/RTT, bytes in flight, min RTT, BBR's estimate
- **Model**: small MLP, 44 → 32 → 32 → 4, ReLU
- **Output**: the 10th, 20th, 30th and 50th **percentile** of the capacity over the next 0.5 s
- **Loss**: quantile (pinball) loss, trained with Adam in numpy
- The scheduler uses the **10th percentile**: "at least this much, 9 times out of 10"

DEPLOYED INSIDE TQUIC

Weights compiled into Rust as constants; two small matrix products per path every 100 ms.

Rust and Python predictions agree to **0.002%** (574 live predictions).

All 566 TQUIC tests pass.

Slide 9. Describe the model: inputs summarise the last second of each path; a small two-layer network outputs percentiles rather than a single guess, trained with the pinball loss which is the standard loss for quantile regression. Using a low percentile builds caution into the estimate instead of a hand-tuned safety margin. It runs inside the transport on the send path and we verified the Rust code matches.

Training without leaking the test data

- **Training traces** (6): Verizon LTE ×2, T-Mobile LTE ×2, AT&T LTE 2016, Verizon EVDO
- **Test traces** (2, never used for training): AT&T LTE, T-Mobile 3G
- **60 emulated sessions**, 20 random conditions (traces scaled to 1–6 Mbit/s, constant rates, outages), 3 schedulers
- **Exact labels**: the emulator knows the true capacity at every moment
- Validated on **4 held-out conditions**; percentile chosen on 4 more

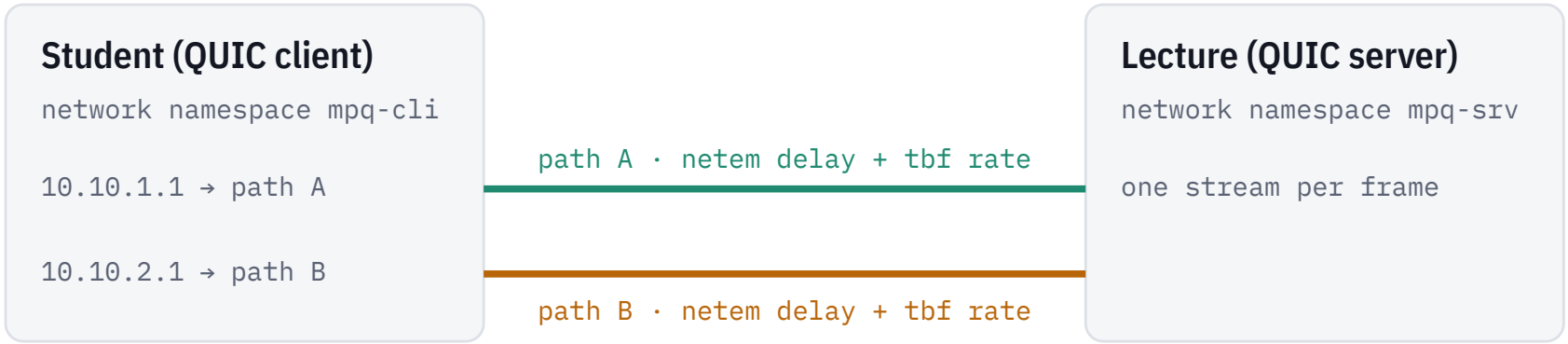
CALIBRATION ON HELD-OUT CONDITIONS

PERCENTILE	TARGET	MEASURED
10th	10%	12.6%
20th	20%	18.9%
30th	30%	25.5%
50th	50%	38.9%

51,210 training and 13,355 validation samples

Slide 10. Emphasise that the model never sees the test traces, and that validation uses whole network conditions it never saw, so the numbers are honest. Labels are exact because in emulation we set the capacity ourselves. The calibration table shows the predicted percentiles behave as percentiles: the 10th percentile is exceeded about 10% of the time.

The testbed: real code, emulated network



- Linux **network namespaces** + **veth** cables; **tc netem** adds delay, **tc tbf** sets rate and a 100 ms queue, in both directions
- A replay script changes each path's rate **every 100 ms** from real cellular traces
- Checked: 40.5 ms measured on a 40 ms path; per-path bytes match TQUIC's own count within 410 bytes

Slide 11. Explain the emulator in one breath: two isolated network stacks on the laptop connected by two virtual cables, each shaped with the kernel's traffic-control tools; a script replays recorded cellular speeds. The real TQUIC code runs on top, so bugs and behaviour are real.

Workload and scenarios

REAL LECTURE

Harvard CopyrightX Lecture 3.2 (W. Fisher), CC BY 3.0, Wikimedia Commons

60 s clip · 1,441 frames at 24 fps · 1.57 Mbit/s · I-frames ~55 KB every 2 s

Encoded like a live stream (H.264, no B-frames, quality-capped)

SCENARIO	PATH A	PATH B
symmetric	4 Mbit/s, 40 ms	4 Mbit/s, 40 ms
rtt_asym	4 Mbit/s, 30 ms	4 Mbit/s, 150 ms
low_bw	1.5 Mbit/s, 60 ms	1 Mbit/s, 120 ms
trace	AT&T LTE trace	T-Mobile 3G trace
outage	4 Mbit/s, stable	off 2 s every 30 s

Slide 12. The video is a real university lecture under an open licence. The five scenarios each test one thing: a sanity check, delay asymmetry, a link that needs both paths, real cellular variability, and a coverage hole. The trace scenario is the realistic one.

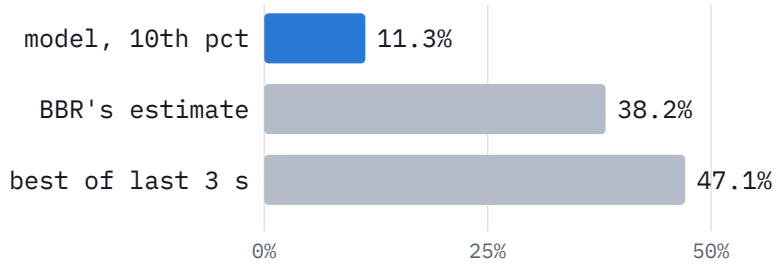
Experiment design

- **8 schedulers**: learned (ours), hand-tuned deadline, frame-pin, MinRTT, ECF, round-robin, redundant, single path
- **5 scenarios × 5 runs** each = **200 runs** of the 60 s lecture
- Runs with the same number see the **same network**, so schedulers are compared **run by run**
- Network rebuilt before every run; all schedulers from **one frozen build**
- Metrics: deadline-miss rate at 150 / 300 / 500 ms, I-frame misses, stall time, frame completion time, bytes per path

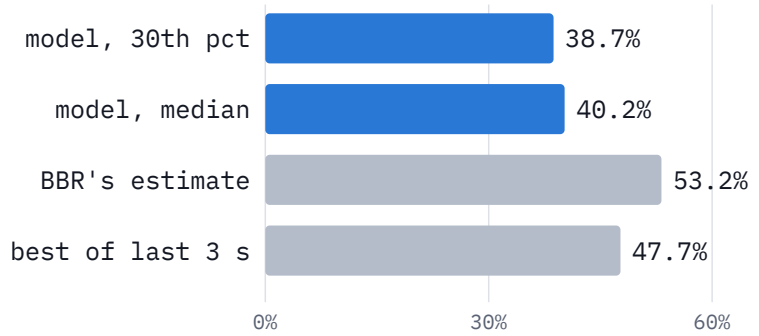
Slide 13. Explain fairness: same network conditions per run number, fresh network each run, one compiled build for everything. Hand-tuned deadline and frame-pin are our ablations. Stall time models a player that waits for late frames.

Result 1: the predictor is the most reliable estimate

How often each estimate is too high (trace)



Median error of the estimate (trace)



31,912 samples on traces the model never saw. Its cautious estimate overestimates a third as often as BBR's, and its estimates are closer to the truth.

Slide 14. This is the main ML result. On the unseen cellular traces, the cautious estimate is too high only about 11% of the time, against 38% for BBR and 47% for the 3-second rule. And the model's own estimates have lower error than BBR's. Mention one caveat: on steady links BBR is already nearly exact.

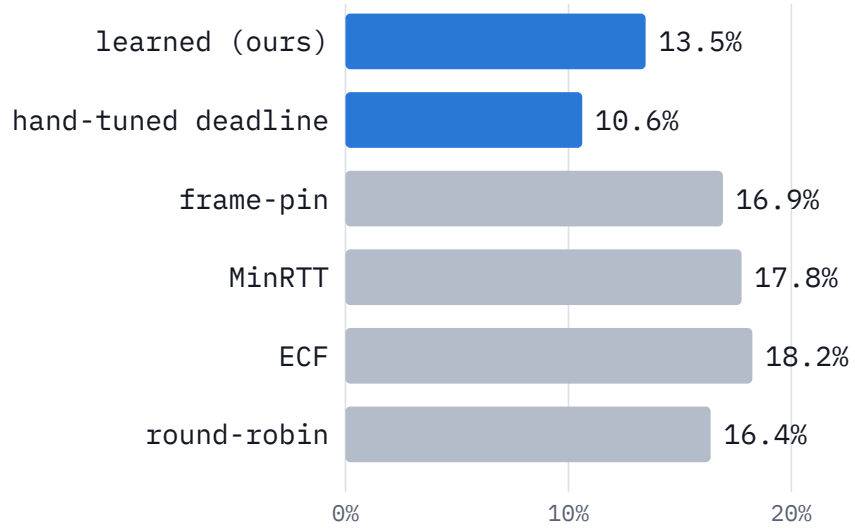
Result 2: deadline misses (300 ms playback delay)

SCHEDULER	SYMMETRIC	RTT_ASYM	LOW_BW	TRACE	OUTAGE
learned (ours)	**0.1%**	**0.3%**	**4.0%**	**13.5%**	**0.6%**
hand-tuned deadline	0.1%	0.1%	3.5%	**10.6%**	0.3%
frame-pin	0.2%	0.1%	3.6%	16.9%	**0.3%**
MinRTT	**0.0%**	**0.0%**	4.7%	17.8%	0.5%
ECF	**0.0%**	**0.0%**	4.9%	18.2%	0.4%
round-robin	**0.0%**	**0.0%**	**3.3%**	16.4%	0.5%
redundant	0.2%	0.1%	99.7%	52.8%	0.5%
single path	0.5%	**0.0%**	99.6%	72.9%	0.3%

Medians over 5 runs. Fast links (symmetric, rtt_asym): almost no misses for anyone. low_bw: big I-frames barely fit. trace: real variability, where scheduling matters.

Slide 15. Read the table by column. On fast links nobody misses. On low_bw, the lecture's big frames barely fit a 300 ms budget for any scheduler; with 500 ms almost everything arrives. On outage everyone is close. The trace column is the interesting one; the next slide zooms in.

Result 3: real cellular traces



redundant 52.8% and single path 72.9% are off the scale.

- Deadline-aware schedulers miss **10.6%** (hand-tuned) and **13.5%** (learned), against **16.4%–18.2%** for transport-only
- Learned vs MinRTT / ECF: fewer misses in 3 and 3 of 5 runs, less stalling in 4 and 4 of 5
- Same order at other delays: 23.2%–24.6% vs 26.9%–28.2% at 150 ms; 5.1%–5.8% vs 7.6%–9.4% at 500 ms

Slide 16. On real variability both deadline-aware schedulers beat every transport-only one, at every playback delay. The paired comparison shows the learned scheduler beats MinRTT and ECF in most runs and stalls less.

What the ablations show

FRAME-PIN (NO DEADLINE CHECK, NO SPLITTING)

Misses **16.9%** on trace, no better than transport-only.

Keeping frames whole is not enough: the **deadline check and splitting** are what help.

LEARNED VS HAND-TUNED ESTIMATE

Hand-tuned 10.6%, learned 13.5%; learned better in 2 of 5 runs.

The better predictor has **not yet** become fewer misses: it is cautious everywhere and sends more traffic over the weaker path.

Slide 17. Be honest here, it is a strength. The frame-pin ablation shows deadline awareness is what matters. The learned scheduler does not yet beat the hand-tuned one even though its predictions are better: its caution costs bandwidth on steady links. That points directly at the next step.

Findings along the way

- **A stall bug in TQUIC**: a path being validated with a full congestion window blocked every other path, ACKs included; whole connections crawled (15 of 1,800 frames in 90 s). Fixed in a few lines; worth reporting upstream.
- **Pinned frames and a rare BBR collapse**: a path fell to ~1 packet/s; frames pinned to it never arrived. Fixed by releasing late frames to any path.
- **Bitrate must fit the network**: at 2.13 Mbit/s the lecture overloaded low_bw and every scheduler missed ~65%; a live platform would lower the bitrate, so we used 1.57 Mbit/s.
- **Measurement pitfall**: WSL2 steps the wall clock by ~0.5 s; all timing uses the monotonic clock.

Slide 18. These show the work was done on real code: we found and fixed a real bug in TQUIC, a robustness problem in our own scheduler, a workload that overloaded the network, and a clock issue that would have corrupted the measurements.

Limitations

- Emulation only: one laptop, public US cellular traces, no live mobile network
- Predictor trained on six public traces; transfer to Indian networks is untested
- One lecture clip, 60 s; five runs per scheduler and scenario
- Two bug fixes in TQUIC apply to every scheduler (all compared on the same build)

Slide 19. Name the limitations before the professor does. Emulation gives control and fairness but not live networks. The model's generalisation to other countries' networks is an open question.

Next steps

1. **Turn better predictions into fewer misses**: less caution on steady paths, e.g. pick the percentile from the predicted spread
2. **Retrain with the learned scheduler's own traffic** (training data so far came from other schedulers)
3. More traces and longer lectures; figures for the final report

Slide 20. The clearest next step comes straight from result 4: the model is accurate but too cautious on steady links. Choosing the percentile adaptively, and retraining on data from the learned scheduler itself, should close the gap with the hand-tuned estimate.

Summary

- Transport-only schedulers miss **16–18%** of lecture frames on real cellular traces
- Deadline-aware frame scheduling cuts that to **10.6%–13.5%**
- Our learned predictor overestimates capacity **11.3%** of the time vs **38.2%** for BBR
- Everything runs in real TQUIC code on an emulated network; code, data and results are reproducible from one repository

Thank you · Questions?

Slide 21. Repeat the three numbers: 16-18% for today's schedulers, about 11-13% with deadline-aware scheduling, and a predictor that is wrong in the dangerous direction a third as often as BBR. Then invite questions.